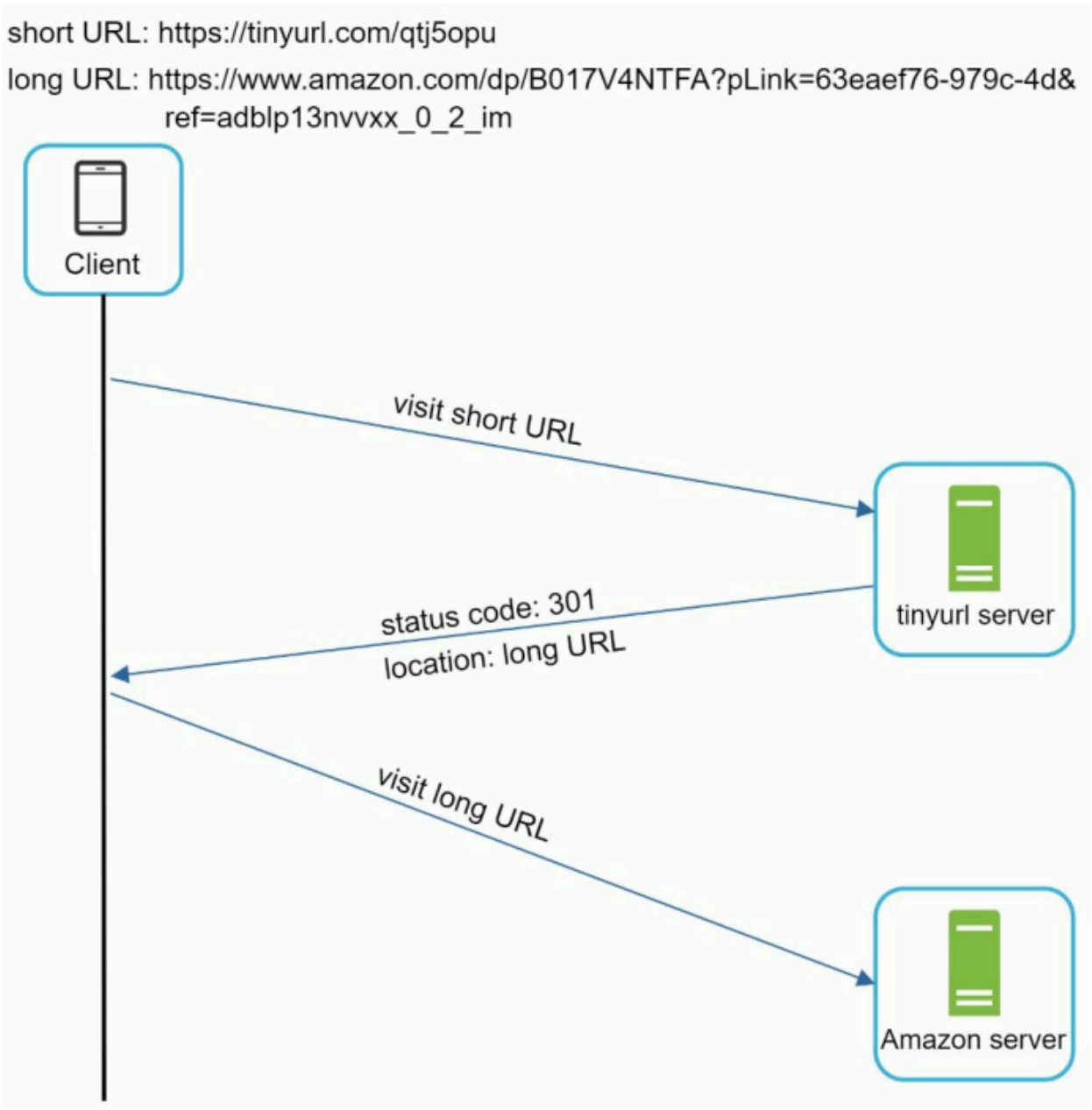
Design a URL Shortener - URL Redirecting

1. Flow Overview



When a user enters a short URL (e.g., from TinyURL) into the browser:

1. Browser sends an **HTTP GET request** to the URL shortener service
2. The server receives the request containing the **short URL key**
3. The service looks up the corresponding **long URL**
4. Server responds with a **redirect response (301 or 302)**
5. Browser automatically redirects the user to the **original long URL**

2. Redirection Types

301 Redirect (Permanent Redirect)

- Indicates that the resource has been **permanently moved**
- Browser **caches the mapping (short → long URL)**

Behavior

- First request:
    - Goes to URL shortener
- Subsequent requests:
    - Go **directly to long URL**
    - URL shortener is bypassed

Advantages

- Reduces **server load significantly**
- Faster response for repeated requests
- Better for **static content**

Disadvantages

- Hard to track analytics after first request
- Cannot easily update destination URL later

302 Redirect (Temporary Redirect)

- Indicates that the resource is **temporarily moved**
- Browser **does NOT cache** the redirect

Behavior

- Every request:
    - Hits URL shortener first
    - Then redirected to long URL

Advantages

- Enables **accurate analytics tracking**
    - Click count
    - User location
    - Device/browser info
- Allows flexibility (destination can change)

Disadvantages

- Increased **latency** due to extra hop
- Higher **server load**

3. Choosing Between 301 vs 302

Factor	301 Redirect	302 Redirect
Caching	Yes	No
Server Load	Low	High
Analytics	Limited	Detailed
Flexibility	Low	High

Decision depends on:

- **Performance priority → 301**
- **Analytics priority → 302**

4. Internal Implementation

Data Structure

- Use a **hash table / key-value store**

```
None
<shortURL, longURL>
```

Lookup Process

```
None
1. Extract shortURL key from request

2. Query storage (cache/database)

3. Retrieve corresponding longURL

4. Return redirect response
```

Example Flow

```
None
Input:  tinyurl.com/abc123

Lookup: abc123 → https://example.com/very/long/url

Output: Redirect to original URL
```

5. Storage Optimization

- Frequently accessed URLs can be stored in:
    - **Cache (e.g., Redis)** → faster lookup

- Less frequently used mappings stored in:
    - **Database**

Improves:

- Latency
- Throughput

6. Edge Considerations

- **Cache miss** → fallback to database
- **Invalid short URL** → return 404
- **High traffic URLs** → cache aggressively
- **Analytics tracking (for 302)** → log request before redirect

7. Key Takeaway

URL redirecting involves:

- Fast lookup of mapping
- Returning appropriate HTTP redirect
- Trade-off between:
    - **Performance (301)**
    - **Analytics & flexibility (302)**